```
import numpy as np
```

```
#inputs
x1,x2 = 0.6,0.1
y = 1

#parameters
w1,w2,b = 0.2,-0.3,0.4
lr = 0.1

#sigmoid
def sigmoid(z):
  return 1/(1+np.exp(-z))


#Forward pass
z = w1*x1+w2*x2+b
y_pred = sigmoid(z)
print(y_pred)

#loss derivative
dL_dy_pred = y_pred-y

#sigmoid derivative
dy_pred_dz = y_pred*(1-y_pred)

#Chain rule
dL_dz = dL_dy_pred*dy_pred_dz

#Gradients
dL_dw1 = dL_dz*x1
dL_dw2 = dL_dz*x2
dL_db = dL_dz

#update
w1 -= lr*dL_dw1
w2 -= lr*dL_dw2
b -= lr*dL_db

print(w1,w2,b)
```

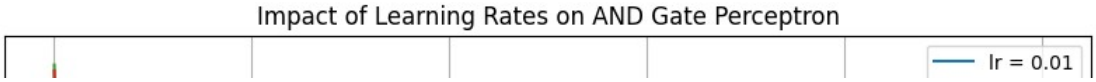
```
0.6201064323430902
0.20536959297953516 -0.2991050678367441 0.4089493216325586
```

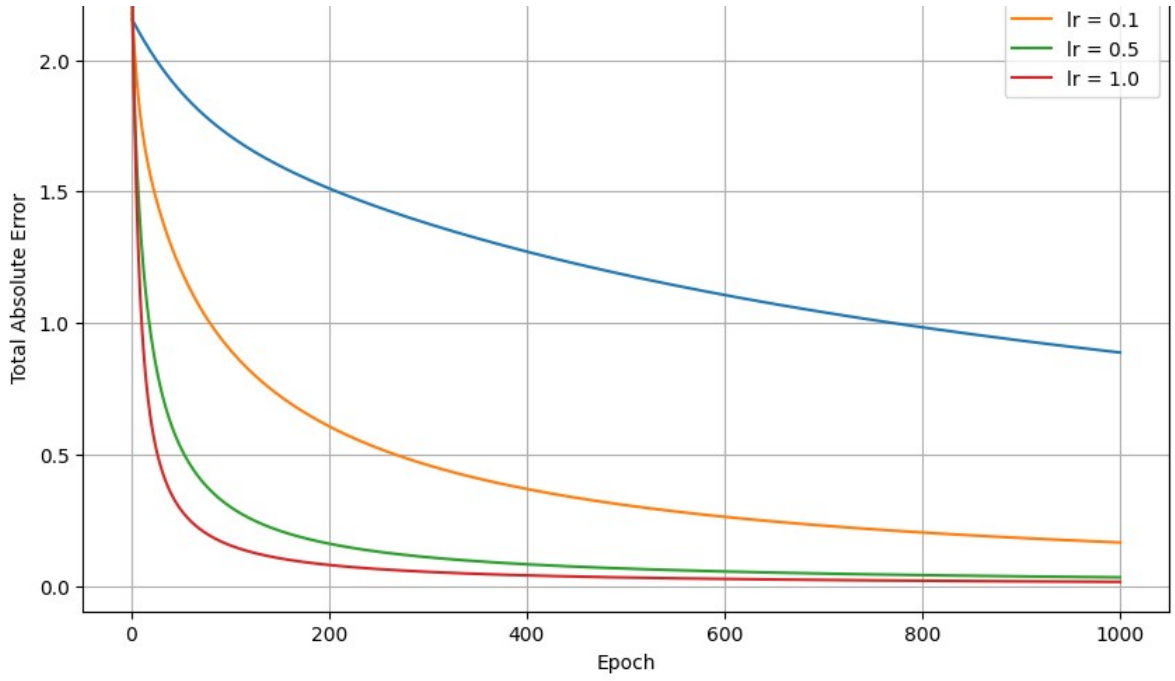
AND Data

```
X = np.array([[0,0],
              [0,1]
```

```
                [0,1],
                [1,0],
                [1,1]])

y = np.array([0,0,0,1])
```

```
#parameters
lr_values = [0.01,0.1,0.5,1.0]


#sigmoid
def sigmoid(z):
  return 1/(1+np.exp(-z))

plt.figure(figsize=(10,6))

for current_lr in lr_values: # Renamed loop variable to avoid shadowing
  w = np.array([0.1,0.2])
  b = 0.3
  errors_per_epoch = []

  for epoch in range(1000):
    total_absolute_error = 0 # Initialize for each epoch
    for i in range(len(X)):
      z = np.dot(X[i],w)+b
      y_pred = sigmoid(z)

      error = y[i] - y_pred
      total_absolute_error += abs(error)

      #Gradient Descent
      dz = y_pred - y[i]
      dw = dz*X[i]
      db = dz

      w -= current_lr*dw # Use current_lr for updates
      b -= current_lr*db # Use current_lr for updates
    errors_per_epoch.append(total_absolute_error)

  # Plotting the results for this specific learning rate
  plt.plot(range(1, 1000 + 1), errors_per_epoch, label=f'lr = {current_lr}') #
plt.title('Impact of Learning Rates on AND Gate Perceptron')
plt.xlabel('Epoch')
plt.ylabel('Total Absolute Error')
plt.legend()
plt.grid(True)
plt.show()
```




lr = 0.1
lr = 0.5
lr = 1.0
Total Absolute Error
2.0
1.5
1.0
0.5
0.0
0
200
400
600
800
1000
Epoch